

Entrega 20% — API REST: Biblioteca Personal

Asignatura: Desarrollo de Aplicaciones Web
Estudiante: Isaias Sánchez
Fecha de entrega: 18 de mayo de 2026

1. Descripción del proyecto

API REST de Biblioteca Personal es una aplicación web completa que permite gestionar una colección personal de libros. El usuario puede registrar libros, llevar el seguimiento de su estado de lectura (pendiente / leyendo / leído), asignar calificaciones del 1 al 5 y escribir notas personales sobre cada libro.

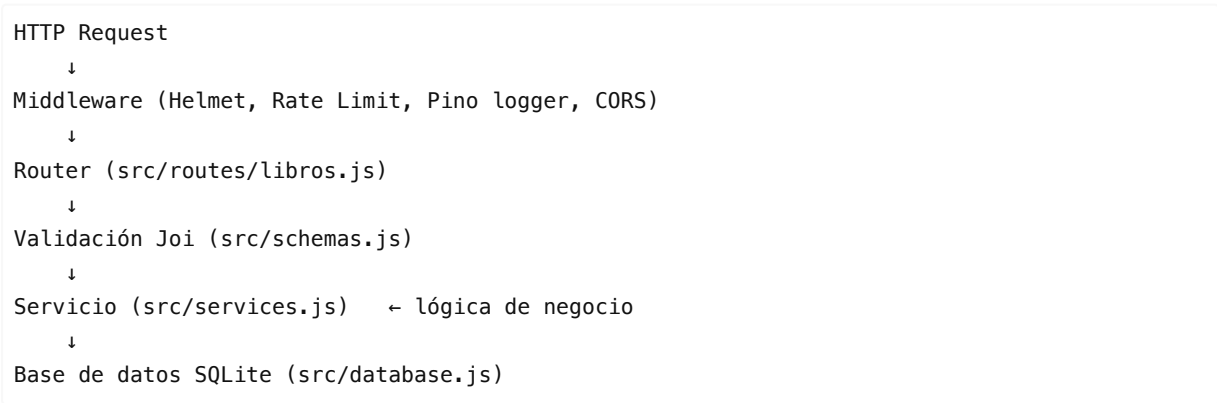
El proyecto incluye tanto el backend (API REST) como un frontend temático de biblioteca, con diseño visual de estantería con libros organizados por colores.

2. Stack tecnológico

Capa	Tecnología	Versión
Runtime	Node.js	18+
Framework	Express	5.x
Base de datos	SQLite (better-sqlite3)	12.x
Validación	Joi	18.x
Seguridad	Helmet + express-rate-limit	—
Logging	Pino	10.x
Documentación	Swagger / OpenAPI 3.0	—
Tests	Jest + Supertest	30.x / 7.x
Deploy	Vercel (serverless)	—
Frontend	HTML5 + CSS3 + JavaScript vanilla	—

3. Arquitectura

El proyecto sigue una arquitectura de capas clara:



Archivos principales:

```

api-biblioteca-personal/
├─ index.js           ← punto de entrada, servidor
├─ seed.js           ← datos de ejemplo (9 libros)
├─ src/
│   ├─ app.js         ← Express + middlewares
│   ├─ database.js    ← conexión SQLite + schema
│   ├─ schemas.js     ← validaciones Joi
│   ├─ services.js    ← lógica de negocio
│   ├─ logger.js      ← logging estructurado (Pino)
│   └─ routes/
│       ├─ libros.js  ← CRUD de libros
│       └─ stats.js   ← estadísticas
├─ public/
│   ├─ index.html     ← frontend biblioteca
│   ├─ style.css       ← estilos (sidebar + panel)
│   └─ app.js         ← lógica frontend
├─ src/__tests__/
│   └─ app.test.js    ← 22 tests de integración
└─ swagger.yaml       ← documentación OpenAPI 3.0

```

4. Endpoints de la API

Libros

Método	Endpoint	Descripción
GET	/libros	Listar libros (filtros: ?estado=, ?genero=, ?page=, ?limit=)
GET	/libros/buscar?q=	Buscar por título o autor (tolerante a acentos)
GET	/libros/:id	Obtener un libro por ID
POST	/libros	Crear un libro
PUT	/libros/:id	Actualizar un libro completo
PATCH	/libros/:id/estado	Cambiar solo el estado de lectura
DELETE	/libros/:id	Eliminar un libro

Estadísticas

Método	Endpoint	Descripción
GET	/stats	Total, por estado, por género, calificación promedio, top 5

5. Modelo de datos

```

{
  "id": 1,
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "genero": "Realismo mágico",
  "anio": 1967,
  "paginas": 471,

```

```
"estado": "leido",
"calificacion": 5,
"notas": "Obra maestra de la literatura latinoamericana",
"creado_en": "2026-05-18 21:32:35",
"actualizado_en": "2026-05-18 21:32:35"
}
```

Estados válidos: pendiente | leyendo | leido

Calificación: entero de 1 a 5 (opcional)

6. Características técnicas destacadas

Seguridad:

- `helmet` — configura automáticamente 14 headers de seguridad HTTP (X-Content-Type-Options, CSP, HSTS, etc.)
- Rate limiting — máximo 100 peticiones por IP cada 15 minutos
- Validación Joi en todos los endpoints — rechaza inputs malformados antes de tocar la DB
- Consultas parametrizadas — previene SQL Injection por diseño

Búsqueda tolerante a acentos:

La búsqueda por título/autor y el filtro por género normalizan los strings con NFD (descomposición canónica Unicode) y eliminan diacríticos antes de comparar. Así `garcia` encuentra `García Márquez` y `distopia` encuentra `Distopía`.

Paginación:

`GET /libros?page=2&limit=5` — respuesta incluye `{libros, paginacion: {total, page, limit, total_paginas}}`.

Logging estructurado:

Cada petición HTTP y cada operación de escritura (crear, actualizar, eliminar) queda registrada con Pino en formato JSON (producción) o legible con colores (desarrollo).

7. Pruebas

El proyecto incluye **22 tests de integración** escritos con Jest y Supertest:

- Cada test usa una base de datos SQLite aislada en `/tmp` que se crea y destruye automáticamente
- Cubren todos los endpoints: listado, búsqueda, obtener por ID, crear, actualizar, cambiar estado, eliminar y estadísticas
- Validan tanto los casos exitosos como los errores (400, 404, validaciones de campos)

Resultado: 22/22 tests pasando

8. Frontend

El frontend incluye:

- **Vista Estantes** — libros representados como espigas de libro en estanterías con colores únicos por género
- **Vista Lista** — tarjetas con metadatos completos
- **Vista Estadísticas** — métricas del catálogo
- **Panel de detalle** — se desliza desde la derecha al hacer clic en un libro; permite cambiar estado y calificación directamente
- **Modal de formulario** — crear y editar libros con validación visual
- **Búsqueda en tiempo real** — tolera acentos, filtra por título y autor simultáneamente
- **Filtros de estado** — sidebar con contadores actualizados en tiempo real

- Animaciones spring — transiciones con `cubic-bezier(0.34, 1.56, 0.64, 1)`

9. Instrucciones de instalación local

```
# Clonar el repositorio
git clone https://github.com/isaias-sanchez/api-biblioteca-personal.git
cd api-biblioteca-personal

# Instalar dependencias
npm install

# Iniciar servidor (la DB se crea automáticamente con 9 libros de ejemplo)
npm start
```

Acceder en: `http://localhost:3000`

10. Links de entrega

Recurso	URL
Repositorio GitHub	https://github.com/isaias-sanchez/api-biblioteca-personal
Demo en producción	https://api-biblioteca-personal-six.vercel.app
Documentación API	https://api-biblioteca-personal-six.vercel.app/api-docs

Proyecto desarrollado como entrega del 20% de la asignatura Desarrollo de Aplicaciones Web — Mayo 2026.